

Pokémon TCG Live – Entity Relationship Justification

Yuhao Liu - A01787782

1. Overview

This database design models the core systems of Pokémon TCG Live, including user accounts, card collections, deck building, matches, progression systems, and cosmetics. The schema ensures data integrity while supporting gameplay features such as competitive matches, inventory tracking, and seasonal progression.

2. Entities and Justification

users

Stores the main account information of each player.

- Attributes:
user_id (PK), email, username, region, created_at
- Constraints:
 - email must be unique and not null
 - username must be unique and not null
- Justification:
This table is the central entity used to identify players across all systems.

profiles

Stores display-related information for users.

- Attributes:
profile_id (PK), user_id (FK), display_name, avatar_id, language, last_login
- Constraints:
 - user_id must be unique (one profile per user)
 - user_id must not be null
 - ON DELETE CASCADE (if a user is deleted, the profile is also removed)

- Justification:
Separates user identity from presentation data.

currencies

Stores in-game currency values.

- Attributes:
currency_id (PK), user_id (FK), coins, crystals, points
- Constraints:
 - coins, crystals, and points must be greater than or equal to 0
 - user_id must not be null
- Justification:
Keeps frequently updated currency data separate.

card_sets

Represents card expansions or sets.

- Attributes:
expansion_id (PK), name, set_code, series, release_date
- Constraints:
 - set_code must be unique and not null
 - release_date must be a valid date
- Justification:
Groups cards into structured sets.

cards

Stores all card definitions.

- Attributes:
card_id (PK), name, card_number, category, rarity, cost, set_id (FK)
- Constraints:
 - card_number and set_id combination must be unique
 - cost must be greater than or equal to 0
- Justification:
Central entity representing all cards.

pokemon_stats

Stores additional attributes for Pokémon cards.

- Attributes:
card_id (PK, FK), hp, type, weakness, resistance, retreat_cost, stage, evolves_from
- Constraints:
 - hp must be greater than 0
 - retreat_cost must be greater than or equal to 0
- Justification:
Extends cards with Pokémon-specific data.

attacks

Defines attacks for Pokémon cards.

- Attributes:
attack_id (PK), card_id (FK), attack_name, damage, energy_cost, description
- Constraints:
 - damage must be greater than or equal to 0
 - card_id must not be null
- Justification:
A card can have multiple attacks.

abilities

Stores special abilities for cards.

- Attributes:
ability_id (PK), card_id (FK), ability_name, description
- Constraints:
 - card_id must not be null
- Justification:
Not all cards have abilities, so this is optional.

inventories

Represents cards owned by users.

- Attributes:
user_id (PK, FK), card_id (PK, FK), quantity, obtained_at
- Constraints:
 - quantity must be greater than or equal to 1
 - user_id and card_id must not be null
 - ON DELETE CASCADE (removes inventory data if user or card is deleted)

- Justification:
Resolves the many-to-many relationship between users and cards.

decks

Stores decks created by users.

- Attributes:
deck_id (PK), user_id (FK), name, format, is_valid, created_at, updated_at
- Constraints:
 - user_id must not be null
- Justification:
Allows players to create and manage decks.

deck_entries

Defines cards included in each deck.

- Attributes:
deck_id (PK, FK), card_id (PK, FK), quantity
- Constraints:
 - quantity must follow valid deck limits
- Justification:
Resolves the many-to-many relationship between decks and cards.

matches

Stores match information.

- Attributes:
match_id (PK), mode, format, start_time, end_time, result
- Constraints:
 - end_time must be after start_time
- Justification:
Keeps match history.

match_players

Links players to matches.

- Attributes:
match_id (PK, FK), user_id (PK, FK), deck_id (FK), is_winner, prizes_taken
- Constraints:
 - each match must have exactly two players
 - prizes_taken must be between 0 and 6
 - ON DELETE CASCADE (removes participants if a match is deleted)
- Justification:
Tracks participation and match results.

cosmetics

Stores cosmetic items.

- Attributes:
cosmetic_id (PK), name, type, rarity
- Constraints:
 - name must not be null
- Justification:
Used for customization features.

user_cosmetics

Tracks cosmetics owned by users.

- Attributes:
user_id (PK, FK), cosmetic_id (PK, FK), obtained_at
- Constraints:
 - user_id and cosmetic_id must not be null
- Justification:
Resolves the many-to-many relationship between users and cosmetics.

seasons

Represents game seasons.

- Attributes:
season_id (PK), name, start_date, end_date
- Constraints:
 - end_date must be after start_date
 - name must not be null
- Justification:
Organizes competitive systems and progression.

rank_progress

Stores ranked progression per season.

- Attributes:
season_id (PK, FK), user_id (PK, FK), points, league, updated_at
- Constraints:
 - points must be greater than or equal to 0
- Justification:
Tracks player ranking performance.

pass_progress

Stores battle pass progress.

- Attributes:
season_id (PK, FK), user_id (PK, FK), tier, xp, is_premium
- Constraints:
 - xp must be greater than or equal to 0
 - tier must be within a valid range
- Justification:
Tracks progression and rewards.

missions

Defines player missions.

- Attributes:
missions_id (PK), type, description, goal, xp_reward, season_id (FK)
- Constraints:
 - goal must be greater than 0
 - xp_reward must be greater than 0
- Justification:
Provides objectives for players.

mission_progress

Tracks mission completion.

- Attributes:
mission_id (PK, FK), user_id (PK, FK), progress, status

- Constraints:
 - progress must be greater than or equal to 0
- Justification:
Resolves the relationship between users and missions.

3. Relationships

User → Profile (1:1)

Each user has exactly one profile, which stores display information such as name and avatar.

User → Decks (1:N)

A user can create multiple decks, while each deck belongs to only one user. This allows players to manage different strategies.

User → Currencies (1:1)

Each user has a single set of currency values used in the game.

Card Set → Cards (1:N)

Each card belongs to one card set, and each set contains multiple cards.

Cards → Pokémon Stats (1:0..1)

Only Pokémon cards have additional attributes such as HP and type, so this relationship is optional.

Cards → Attacks (1:N)

A card can have one or more attacks.

Cards → Abilities (1:0..N)

A card may have zero or multiple abilities.

Users ↔ Cards (N:M via inventories)

Users and cards are connected through the inventories table.

A user can own multiple cards, and each card can belong to multiple users.

Decks ↔ Cards (N:M via deck_entries)

Decks and cards are related through `deck_entries`.

A deck contains multiple cards, and a card can appear in multiple decks.

Match → Players (1:N)

Each match includes exactly two players, stored in the `match_players` table.

User → Match Participation (1:N)

A user can participate in multiple matches over time.

Deck → Match Players (1:0..N)

A deck can be used in multiple matches, but it is not required to be used.

Season → Rank Progress (1:N)

Each season stores ranking data for multiple users.

Season → Pass Progress (1:N)

Each season contains battle pass progress for users.

Season → Missions (1:N)

A season defines multiple missions.

Mission → Mission Progress (1:N)

Each mission has progress records for different users.

User → Mission Progress (1:N)

A user can have progress in multiple missions.

Users ↔ Cosmetics (N:M via `user_cosmetics`)

Users can own multiple cosmetic items, and each cosmetic item can belong to multiple users.